

CLAASP-MP: An Automated MILP Framework for Monomial Prediction

Emanuele Bellini, Mohamed Rachidi and Sharwan K. Tiwari

Cryptography Research Center, Technology Innovation Institute, Abu Dhabi, UAE
\{emanuele.bellini,mohamed.rachidi,sharwan.tiwari\}@tii.ae

Abstract. We present CLAASP-MP, a monomial prediction (MP) tool based on the three-subset division property without unknown subset (3SDP-woU), integrated into CLAASP, a library for automated analysis of symmetric key cryptographic primitives. The propagation rules of 3SDP-woU are encoded as a Mixed Integer Linear Programming (MILP) model generated directly from the CLAASP component graph, covering the main building blocks used in modern symmetric designs, including S-boxes, linear layers, Boolean word operations, modular addition and multiplication, and linear and nonlinear feedback shift register updates.

Using this model for selected output bits, CLAASP-MP computes algebraic normal form (ANF) (for a small number of rounds), derives superpolies for a chosen cube, and computes a tight upper bound on the algebraic degree with respect to plaintext, key, or IV variables. A monomial is absent if its ANF coefficient is zero, if it does not appear in the superpoly, or if the algebraic degree is too small for it to occur; in all such cases, the output is balanced and yields an integral distinguisher.

We evaluate CLAASP-MP on block ciphers, permutations, and stream ciphers implemented in CLAASP. We introduce an exact bit-level MILP modeling technique for modular multiplication and apply it to provide the first independent algebraic analysis of the MSX block cipher, identifying integral properties that persist up to 7 rounds for MSX-128. Furthermore, we reproduce known distinguishers and identify new cubes for several block ciphers, and extend the best-known integral distinguishers of ChaCha and Salsa permutations from 6 and 5 rounds, respectively, to 6.75 rounds. We also discovered new integral distinguishers for Bivium using several 36-dimensional cubes and extended the exact computation of the algebraic degree of Trivium to later initialization clocks, while reproducing many published superpolies.

These results show that CLAASP-MP provides a unified and practical MILP-based framework for monomial prediction and algebraic analysis across a wide range of symmetric primitives.

Keywords: Division property, Integral distinguishers, Superpoly, Cube attacks, Monomial Prediction

1 Introduction

Symmetric cryptanalysis often exploits the algebraic structure of a cipher, for example, by studying which monomials appear in the algebraic normal form (ANF) of its output bits and how the algebraic degree evolves across rounds. Such information is central to integral and cube attacks, as well as degree-based distinguishers. However, for realistic primitives, directly expanding ANFs or tracking monomials quickly becomes infeasible.

The *division property*, introduced by Todo at Eurocrypt 2015 [Tod15a], provides a principled way to track monomial propagation. It generalizes the classical *integral property* of Daemen et al. [DKR97], where one studies the XOR of ciphertext bits over a structured set of plaintexts (a *cube*).

When the output bit of a cipher is examined over a multidimensional cube of input values, the combined result of all evaluations on that cube is guaranteed to cancel out to zero as long as the algebraic degree of that output bit is lower than the dimension of the cube. In formulas, if the cipher output bit $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is evaluated over a cube $\mathcal{C}$, then $\bigoplus_{x \in \mathcal{C}} f(x) = 0$ whenever $\deg(f) < \dim(\mathcal{C})$. When the sum is nonzero, the result is a key-dependent polynomial, called *superpoly*, that can be exploited for key recovery attacks.

Todo’s *two-subset division property* [Tod15b] formalizes this phenomenon by tracking sets of monomials across rounds. It enabled improved integral distinguishers and led to the first theoretical analysis of full-round MISTY1 [Tod15a]. Todo and Morii [TM16] extended this to the *bit-based* and *three-subset* formulation, offering finer monomial tracking and tighter algebraic degree bounds. Boura and Canteaut [BC16] provided an equivalent parity-set view.

Direct propagation of the bit-based or three-subset division property becomes infeasible as the number of rounds or the state size increases. Xiang et al. [XZBL16] addressed this by encoding these propagation rules as linear constraints in a MILP framework, enabling solver-driven searches for integral distinguishers. Later refinements introduced simplified propagation rules [HW19] and pruning strategies [WHG⁺19].

A key simplification came from Hao et al. [HLM⁺20], who showed that when key bits are treated as inputs, the “unknown” subset in the three-subset formulation can be removed without loss for public functions, yielding the three-subset division property *without unknown subset* (3SDP-woU). Building on this, Hu et al. [HSWW20] introduced *monomial prediction* (MP), which determines whether an input monomial contributes to a target output monomial via the parity of monomial trails. Recent works have further strengthened MP by improving the modeling of nonlinear components such as modular addition, as well as by developing nested monomial prediction and core monomial prediction techniques [HST⁺21, DL22, HHPW22, HHLW24, HY24].

1.1 Related Works

We now provide a brief review of prior work on automating division-property analysis and constructing practical tools. Early attempts to mechanize division property propagation include the MILP formulation of Xiang et al. [XZBL16], later refined by simplified rules [HW19], pruning techniques [WHG⁺19], and the 3SDP-woU formulation [HLM⁺20].

A closely related automated framework is the SAT-based tool of Eskandari et al. [EKK18], which models division property transitions as propositional constraints and searches for integral distinguishers. This approach is effective but focuses on integrals and constructs a new SAT instance for each cube. More details on how this framework compares to our work are provided in Section 5.

Recent advances in monomial prediction [HST⁺21, DL22, HHPW22, HHLW24, HY24] have further extended the reach of the analysis by improving the modeling of nonlinear components and by introducing nested monomial prediction and core monomial prediction techniques.

Overall, existing tools either target integral distinguishers or rely on component-wise, cipher-specific encodings. This motivates the development of unified frameworks that support ANF extraction, degree analysis, and cube-based attacks within one modeling abstraction.

1.2 Our Contribution

In this work, we introduce a generic and fully automated MILP framework integrated into the CLAASP library [BGG⁺23] (<https://github.com/Crypto-TII/claasp>), based on the 3SDP-woU formulation and monomial prediction. From the cipher’s internal DAG representation, it automatically generates linear constraints for core components, including

S-boxes, linear layers, Boolean word operations, modular addition and multiplication, and linear and nonlinear FSR components. A new MILP modeling technique for modular multiplication is introduced, based on its exact bit-level algebraic expansion over $\mathbb{Z}/2^n\mathbb{Z}$, whereas previous approaches relied on SAT encodings derived from Karnaugh-map-based Boolean representations of the multiplication.

Using this unified model, CLAASP-MP computes for a selected output bit (i) algebraic normal forms, (ii) superpolies for chosen cubes, and (iii) tight algebraic degree bounds in key, plaintext, or IV variables. The same formulation applies to block ciphers, stream ciphers, hash functions, and permutations.

Our experiments cover several such primitives available in CLAASP. Besides reproducing previously known integral distinguishers and identifying new cubes for several block ciphers, we demonstrate the effectiveness of our new modular multiplication model and of the CLAASP-MP framework by applying it to the MSX family of lightweight block ciphers [MSR⁺26]. We provide the first independent algebraic analysis of MSX and derive tighter and more fine-grained upper bounds on its algebraic degree. In contrast to the designers' global degree-growth analysis, our automated framework identifies bit- and word-level structural properties, reveals earlier degree saturation for MSX-64, and exposes persistent integral properties in MSX-128 up to 7 rounds. Moreover, we translate these bounds into explicit integral distinguishers, demonstrating concrete cryptanalytic implications beyond prior analysis. We also use CLAASP-MP to extend the best-known distinguishers of the ChaCha and Salsa permutations from 6 rounds to 6.75 and from 5 to 6.75 rounds, respectively. For stream ciphers, CLAASP-MP finds many new 36-dimensional integral cubes for Bivium and extends the exact algebraic degree computation of Trivium to later initialization clocks while reproducing many known superpolies. Finally, we compare our approach with the SAT-based tool Solvatore and provide practical implementation examples in the Appendix.

These results show that CLAASP-MP provides an automated MILP framework for monomial prediction and algebraic analysis across a wide range of symmetric primitives.

We emphasize that, in this work, our objective is not to derive new cryptanalytic results using the 3SDP-woU technique, but rather to design a fully automated and extensible analysis framework; in practice, obtaining improved distinguishers with this methodology typically requires extensive MILP-based searches over carefully constructed cube selections, a process that may span several weeks of computational effort. We hope this tool will serve as a basis for finding improved distinguishers in subsequent works.

1.3 Outline

The paper is organized as follows. Preliminaries about division property, monomial prediction, algebraic attacks, and MILP modeling for propagation rules are provided in Section 2. Section 3 describes CLAASP-MP, and how the MILP model is constructed from the CLAASP component graph. Section 4 reports experimental results on block ciphers, permutations, and stream ciphers. Section 5 discusses differences between our MILP framework and a SAT-based approach. Implementation details and usage examples appear in the Appendix.

2 Preliminaries

2.1 Notations

Let $\mathbb{F}_2 = \{0, 1\}$ and let $\mathbb{F}_2^n$ denote the n -dimensional vector space over $\mathbb{F}_2$. For vectors $\mathbf{x} = (x_0, \dots, x_{n-1})$ and $\mathbf{y} = (y_0, \dots, y_{n-1})$ in $\mathbb{F}_2^n$, we write $\mathbf{x} \preceq \mathbf{y}$ if $x_i \leq y_i$ for all i , and

$\mathbf{x} \succeq \mathbf{y}$ if $x_i \geq y_i$ for all i . The Hamming weight of $\mathbf{x}$, denoted by $\text{wt}(\mathbf{x})$, is the number of ones in $\mathbf{x}$.

For a Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, its algebraic normal form (ANF) is defined as

$$f(\mathbf{x}) = \bigoplus_{\mathbf{u} \in \mathbb{F}_2^n} a_{\mathbf{u}} \mathbf{x}^{\mathbf{u}}, \quad a_{\mathbf{u}} \in \mathbb{F}_2,$$

where $\mathbf{x}^{\mathbf{u}} = \prod_{i=0}^{n-1} x_i^{u_i}$ denotes the *monomial* corresponding to $\mathbf{u}$. The coefficient $a_{\mathbf{u}}$ can be expressed as

$$a_{\mathbf{u}} = \bigoplus_{\mathbf{x} \preceq \mathbf{u}} f(\mathbf{x}),$$

and the algebraic degree of f is $\deg(f) = \max_{\mathbf{u}: a_{\mathbf{u}}=1} \text{wt}(\mathbf{u})$. It is convenient to denote a monomial function by

$$\pi_{\mathbf{u}}(\mathbf{x}) = \mathbf{x}^{\mathbf{u}} = \prod_{i=0}^{n-1} x_i^{u_i},$$

so that the ANF of f is the XOR of all $\pi_{\mathbf{u}}$ with $a_{\mathbf{u}} = 1$.

2.2 Division property

The division property, introduced by Todo [Tod15b], traces how monomials propagate through the round functions of a cipher and generalizes integral cryptanalysis. It provides a structured way to reason about the presence or absence of monomials in the ANF of an output bit.

Let $\mathbb{X} \subseteq \mathbb{F}_2^n$ be a multiset of input vectors. For a binary vector $\mathbf{u} \in \mathbb{F}_2^n$, the parity of the monomial $\pi_{\mathbf{u}}(\mathbf{x})$ over $\mathbb{X}$ is

$$P_{\mathbf{u}} = \bigoplus_{\mathbf{x} \in \mathbb{X}} \pi_{\mathbf{u}}(\mathbf{x}) = \bigoplus_{\mathbf{x} \in \mathbb{X}} \mathbf{x}^{\mathbf{u}}.$$

If $P_{\mathbf{u}} = 0$, the contribution of $\mathbf{x}^{\mathbf{u}}$ cancels in the sum; if $P_{\mathbf{u}} = 1$, the monomial $\mathbf{x}^{\mathbf{u}}$ is active. For any Boolean function $f(\mathbf{x}) = \sum_{\mathbf{u}} a_{\mathbf{u}} \mathbf{x}^{\mathbf{u}}$, we have

$$\bigoplus_{\mathbf{x} \in \mathbb{X}} f(\mathbf{x}) = \sum_{\mathbf{u}} a_{\mathbf{u}}^f P_{\mathbf{u}},$$

so the effect of each coefficient $a_{\mathbf{u}}^f$ on the XOR over $\mathbb{X}$ is fully determined by the parities $P_{\mathbf{u}}$. If the division property ensures that certain $P_{\mathbf{u}}$ are always zero, then the corresponding ANF coefficients must vanish. Conversely, if $P_{\mathbf{u}}$ may be nonzero, the monomial $\mathbf{x}^{\mathbf{u}}$ can influence the output. This allows us to reason which monomials can or cannot appear in the ANF of complex Boolean functions.

2.2.1 Two-subset bit-based division property.

In the bit-based division property [TM16], each exponent vector $\mathbf{u}$ belongs either to a set of monomials with unknown parity or to the set of monomials whose parity is guaranteed to be zero. Let $\mathbb{K}$ be the set of exponent vectors with unknown parity. The multiset $\mathbb{X}$ is said to satisfy the two-subset division property $D_{\mathbb{K}}^{1^n}$ if

$$P_{\mathbf{u}} = \bigoplus_{\mathbf{x} \in \mathbb{X}} \pi_{\mathbf{u}}(\mathbf{x}) = \begin{cases} \text{unknown,} & \text{if there exists } \mathbf{k} \in \mathbb{K} \text{ such that } \mathbf{u} \succeq \mathbf{k}, \\ 0, & \text{otherwise.} \end{cases}$$

For a multiset of chosen plaintexts (or IVs), one first determines its division property $D_{\mathbb{K}}^{1^n}$ and then propagates $\mathbb{K}$ through each round function. integral distinguishers are detected

by checking whether certain unit vectors are missing in the propagated division set. Let $\mathbb{K}$ propagate to $\mathbb{K}'$ and let e_i be the unit vector corresponding to an output bit y_i . If $e_i \notin \mathbb{K}'$, then by definition no $\mathbf{k} \in \mathbb{K}'$ satisfies $e_i \succeq \mathbf{k}$, so the parity P_{e_i} must be zero. Since $\pi_{e_i}(\mathbf{y}) = y_i$, we obtain

$$\bigoplus_{\mathbf{x} \in \mathbb{X}} y_i = 0,$$

i.e., y_i is balanced over the chosen input structure. If all unit vectors appear in $\mathbb{K}'$, no output bit is guaranteed to be balanced.

Another important algebraic consequence [TIHM18] of the division property arises when analyzing a public function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$. Let $\mathbb{X}$ be a cube $\mathcal{C}_I$ with $D_{\{\mathbf{k}\}}^{1^n}$, and assume no division trail from the exponent $\mathbf{k}$ reaches to the unit vector $\mathbf{1}$, then

$$\bigoplus_{\mathbf{x} \in \mathbb{X}} f(\mathbf{x}) = 0.$$

Thus,

$$0 = \bigoplus_{\mathbf{x} \in \mathbb{X}} f(\mathbf{x}) = \sum_{\mathbf{u}} a_{\mathbf{u}} P_{\mathbf{u}} = \sum_{\mathbf{u} \succeq \mathbf{k}} a_{\mathbf{u}} P_{\mathbf{u}} + \sum_{\mathbf{u} \not\succeq \mathbf{k}} a_{\mathbf{u}} P_{\mathbf{u}}.$$

The second term is zero because $P_{\mathbf{u}} = 0$ for all $\mathbf{u} \not\succeq \mathbf{k}$, so

$$0 = \sum_{\mathbf{u} \succeq \mathbf{k}} a_{\mathbf{u}} P_{\mathbf{u}}.$$

For $\mathbf{u} \succeq \mathbf{k}$, $P_{\mathbf{u}}$ depends on the fixed non-cube variables, and the equality must hold for every choice of these constants. Therefore,

$$a_{\mathbf{u}} = 0, \text{ for all } \mathbf{u} \succeq \mathbf{k}.$$

2.2.2 Three-subset bit-based division property.

The three-subset division property [WHG⁺19] refines this by distinguishing monomials of odd parity in addition to those of even parity. Instead of classifying monomials only as *unknown* or *always even*, it introduces a class for monomials whose parity is guaranteed to be *odd*. This extra information allows certain integral properties to be detected that may be missed under the two-subset formulation.

Let $\mathbb{K}, \mathbb{L} \subseteq \mathbb{F}_2^n$ be two sets of exponent vectors, where $\mathbb{K}$ contains the monomials whose parity is unknown and $\mathbb{L}$ contains those whose parity is one. A multiset $\mathbb{X} \subseteq \mathbb{F}_2^n$ is said to satisfy the three-subset division property $D_{\mathbb{K}, \mathbb{L}}^{1^n}$ if

$$P_{\mathbf{u}} = \bigoplus_{\mathbf{x} \in \mathbb{X}} \pi_{\mathbf{u}}(\mathbf{x}) = \begin{cases} \text{unknown,} & \text{if there exists } \mathbf{k} \in \mathbb{K} \text{ with } \mathbf{u} \succeq \mathbf{k}, \\ 1, & \text{if there exists } \mathbf{l} \in \mathbb{L} \text{ with } \mathbf{u} = \mathbf{l}, \\ 0, & \text{otherwise.} \end{cases}$$

This separates monomials that are always even, always odd, or undetermined, giving more precise information than the two-subset version. For example, the three-subset property was essential in proving that the experimentally observed 15-round integral property of SIMON 32 holds for all keys.

For a public Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, the unknown subset $\mathbb{K}$ and odd-parity subset $\mathbb{L}$ propagate independently. The authors of [WHG⁺19] showed that odd parity information alone can determine the presence of specific monomials in the ANF of f . A simplified statement (see [HLM⁺20], is: let $f(\mathbf{x}) = \sum_{\mathbf{u}} a_{\mathbf{u}} \mathbf{x}^{\mathbf{u}}$ and let ℓ be an n -dimensional bit vector. If the initial division property $D_{\emptyset, \{\ell\}}^{1^n}$ propagates through f to $D_{\emptyset, \{\ell\}}^1$, then $a_{\ell} = 1$. Thus, the three-subset property can determine the presence of specific ANF monomials.

2.2.3 Three-subset division property without unknown subset (3SDP-woU)

In the original three-subset division property, the unknown subset $\mathbb{K}$ represents monomials whose parity depends on secret key bits, and key addition introduces new unknowns at every round. This makes it difficult to model three-subset propagation in MILP or SMT frameworks. Hu and Wang [HW19] proposed a variant of the three-subset with simplified propagation rules, while Wang *et al.* [WHG⁺19] combined MILP modeling with a pruning technique to improve accuracy. However, both approaches become computationally demanding for large ciphers.

For public Boolean functions, however, the unknown subset is unnecessary. Hao *et al.* [HLM⁺20] observed that if the round function and the key schedule are both treated as part of the public mapping, then key bits can be regarded as input variables. Every monomial then has a fixed ANF coefficient, and no parity uncertainty arises. Thus, the unknown subset can be removed without loss of accuracy, leading to the *three-subset division property without unknown subset* (3SDP-woU).

Let $\mathbb{X} \subseteq \mathbb{F}_2^n$ be a multiset of input vectors and let $\tilde{\mathbb{L}}$ be a multiset of n -bit exponent vectors. The set $\mathbb{X}$ is said to satisfy the 3SDP-woU, denoted $D_{\tilde{\mathbb{L}}}^{\mathbb{X}}$, if for each $\mathbf{u} \in \mathbb{F}_2^n$,

$$P_{\mathbf{u}} = \bigoplus_{\mathbf{x} \in \mathbb{X}} \pi_{\mathbf{u}}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{u} \text{ appears an odd number of times in } \tilde{\mathbb{L}}, \\ 0, & \text{otherwise.} \end{cases}$$

This key-independent formulation allows the propagation of monomial parities to be modeled purely algebraically, enabling automatic analysis through MILP constraints in our monomial prediction framework.

2.3 Monomial prediction

Monomial prediction (MP) provides an algebraic formulation of 3SDP-woU by characterizing when an input monomial contributes to a given output monomial.

Let $\mathbf{f} : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be a vectorial Boolean function, $\mathbf{f}(\mathbf{x}) = (f_0(\mathbf{x}), \dots, f_{m-1}(\mathbf{x}))$. For $\mathbf{v} = (v_0, \dots, v_{m-1}) \in \mathbb{F}_2^m$,

$$\mathbf{y}^{\mathbf{v}} = \prod_{i=0}^{m-1} f_i(\mathbf{x})^{v_i}.$$

Expanding $\mathbf{y}^{\mathbf{v}}$ in the input variables $\mathbf{x}$ gives

$$\mathbf{y}^{\mathbf{v}} = \bigoplus_{\mathbf{u} \in \mathbb{F}_2^n} a_{\mathbf{u}, \mathbf{v}} \mathbf{x}^{\mathbf{u}}, \quad a_{\mathbf{u}, \mathbf{v}} \in \mathbb{F}_2,$$

where $a_{\mathbf{u}, \mathbf{v}} = 1$ indicates that $\mathbf{x}^{\mathbf{u}}$ appears in the ANF of $\mathbf{y}^{\mathbf{v}}$, denoted $\mathbf{x}^{\mathbf{u}} \rightarrow \mathbf{y}^{\mathbf{v}}$. The goal of MP is to decide, for given $\mathbf{u}$ and $\mathbf{v}$, whether $a_{\mathbf{u}, \mathbf{v}} = 1$.

Monomial trails. In practice, $\mathbf{f}$ is given as the composition of simpler round (or layer) functions:

$$\mathbf{f} = \mathbf{f}^{(r-1)} \circ \mathbf{f}^{(r-2)} \circ \dots \circ \mathbf{f}^{(0)},$$

where each $\mathbf{f}^{(i)} : \mathbb{F}_2^{n_i} \rightarrow \mathbb{F}_2^{n_{i+1}}$ is a simple (round) operation of the cipher. Let $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(i+1)}$ denote the input and output of $\mathbf{f}^{(i)}$ so that $\mathbf{x}^{(i+1)} = \mathbf{f}^{(i)}(\mathbf{x}^{(i)})$.

Definition 1. A sequence of monomials

$$(\mathbf{x}^{(0)})^{\mathbf{u}^{(0)}} \rightarrow (\mathbf{x}^{(1)})^{\mathbf{u}^{(1)}} \rightarrow \dots \rightarrow (\mathbf{x}^{(r)})^{\mathbf{u}^{(r)}}$$

is called an *r-round monomial trail* from $(\mathbf{x}^{(0)})^{\mathbf{u}^{(0)}}$ to $(\mathbf{x}^{(r)})^{\mathbf{u}^{(r)}}$ if for each $i = 0, \dots, r-1$ the propagation $(\mathbf{x}^{(i)})^{\mathbf{u}^{(i)}} \rightarrow (\mathbf{x}^{(i+1)})^{\mathbf{u}^{(i+1)}}$ is valid according to the algebraic expansion of $\mathbf{f}^{(i)}$. We write $(\mathbf{x}^{(0)})^{\mathbf{u}^{(0)}} \rightsquigarrow (\mathbf{x}^{(r)})^{\mathbf{u}^{(r)}}$ if at least one such trail exists.

A valid trail indicates that the input monomial may contribute to the given output monomial. Since different trails may lead to the same output monomial, it is the *parity* of the number of trails that determines whether the monomial actually appears in the ANF.

2.3.1 Monomial hull and parity.

Let $\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(r)})$ denote the set of all trails connecting $\mathbf{x}^{\mathbf{u}^{(0)}}$ and $\mathbf{x}^{\mathbf{u}^{(r)}}$:

$$|\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(r)})| = \#\{\text{valid trails } \mathbf{x}^{\mathbf{u}^{(0)}} \rightsquigarrow \mathbf{x}^{\mathbf{u}^{(r)}}\}.$$

The parity determines the ANF coefficient:

$$a_{\mathbf{u}^{(0)}, \mathbf{u}^{(r)}} = |\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(r)})| \bmod 2,$$

so $\mathbf{x}^{\mathbf{u}^{(0)}}$ appears in the ANF of $\mathbf{x}^{\mathbf{u}^{(r)}}$ precisely when the number of trails is odd. The number of trails satisfies the recursion

$$|\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(i)})| = \sum_{\substack{\mathbf{u}^{(i-1)}: \\ (\mathbf{x}^{(i-1)})^{\mathbf{u}^{(i-1)}} \rightarrow (\mathbf{x}^{(i)})^{\mathbf{u}^{(i)}}}} |\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(i-1)})|.$$

Proposition 1 (Monomial prediction criterion [HSWW20]). *Let $\mathbf{f} = \mathbf{f}^{(r-1)} \circ \dots \circ \mathbf{f}^{(0)}$. Then*

$$(\mathbf{x}^{(0)})^{\mathbf{u}^{(0)}} \rightarrow (\mathbf{x}^{(r)})^{\mathbf{u}^{(r)}} \iff |\text{Hull}(\mathbf{u}^{(0)}, \mathbf{u}^{(r)})| \equiv 1 \pmod{2}.$$

This criterion links the presence of an ANF monomial to the parity of trails connecting the input and output exponents. In the 3SDP-woU framework, this parity propagation is expressed algebraically and captured by the MILP constraints used in our implementation.

We conclude this section by briefly indicating how these notions are used in algebraic attacks.

2.4 Algebraic attacks based on the division property and monomial prediction

Let an output bit of a symmetric cipher be represented as a Boolean function

$$f : \mathbb{F}_2^n \times \mathbb{F}_2^m \rightarrow \mathbb{F}_2, \quad (\mathbf{x}, \mathbf{k}) \mapsto f(\mathbf{x}, \mathbf{k}),$$

where $\mathbf{x}$ denotes public inputs (plaintext or IV) and $\mathbf{k}$ the key bits. Its ANF can be written as

$$f(\mathbf{x}, \mathbf{k}) = \sum_{u \in \{0,1\}^n, v \in \{0,1\}^m} a_{u,v} \mathbf{x}^u \mathbf{k}^v.$$

Fix a cube $\mathcal{C}_I \subseteq \mathbb{F}_2^n$, an affine subspace determined by a set of public coordinates I . Split $\mathbf{x} = (\mathbf{x}_I, \mathbf{x}_{\bar{I}})$, and fix $\mathbf{x}_{\bar{I}} = \mathbf{c}$. Then f can be uniquely written as

$$f(\mathbf{x}_I, \mathbf{c}, \mathbf{k}) = S_{\mathcal{C}_I}(\mathbf{k}) \mathbf{x}_I + Q(\mathbf{x}_I, \mathbf{k}),$$

where $\mathbf{x}_I = \prod_{i \in I} x_i$ and Q is not divisible by $\mathbf{x}_I$. Summing over the cube eliminates all terms of Q and yields

$$\bigoplus_{\mathbf{x}_I \in \mathcal{C}_I} f(\mathbf{x}_I, \mathbf{c}, \mathbf{k}) = S_{\mathcal{C}_I}(\mathbf{k}).$$

If $S_{\mathcal{C}_I}(\mathbf{k}) \equiv 0$ for all keys, the cube is *balanced*, giving an integral distinguisher. If $S_{\mathcal{C}_I}(\mathbf{k})$ is low degree or sparse and $\mathbf{c}$ fixed (commonly $\mathbf{c} = 0$), then evaluating the cipher on $\mathcal{C}_I$ provide relations in the key variables. When $S_{\mathcal{C}_I}$ is linear, these relations can directly

reveal key bits [DS09]. A main practical task is to choose cubes for which $S_{C_I}(k)$ is simple, or to locate outputs whose degree in x remains low after several rounds; this cube and target selection problem is precisely what the division property and monomial prediction are designed to address.

If the algebraic degree of $f(\mathbf{x}, \mathbf{k})$ with respect to $\mathbf{x}$ is at most d then the XOR of f over any affine subspace of dimension $d + 1$ vanishes: $\bigoplus_{\mathbf{x} \in \mathcal{A}} f(\mathbf{x}, \mathbf{k}) = 0$. This principle underlies integral and higher-order differential attacks [DKR97, Knu94]. Degree-based distinguishers use it to estimate the number of secure rounds and to detect degree drops caused by cancellations. Division property based degree bounds are typically tighter than classical ones [BC13], since they track the evolution of individual monomials rather than only the global degree. If at some intermediate state of a block cipher, many output bits are balanced over a low-dimensional affine space $\mathcal{A}$, one can often append extra rounds and extend the distinguisher into a partial key recovery attack. For stream ciphers, balanced keystream bits yield distinguishers and cannot be used for key recovery.

An alternative approach within algebraic cryptanalysis models the cipher as a system of equations in plaintext and ciphertext bits and attempts to recover key variables using Gröbner basis, XL, or SAT-solving techniques [ST22, SPTV22, SPTV24]. These methods usually require only a few known plaintext–ciphertext pairs but are computationally demanding.

The division property and its monomial prediction provide algebraic rules to determine, without expanding the full ANF, which input monomials can reach a chosen output or contribute to the parity of outputs over a cube. The MILP framework CLAASP-MP implements these rules under 3SDP-woU, enabling automated algebraic analysis across different primitives.

2.5 Propagation Rules and MILP Modeling of Division Property

The monomial prediction framework requires a precise description of how basic Boolean operations affect the propagation of monomials between their input and output variables. Under the 3SDP-woU framework, the propagation rules for the basic Boolean operations form the basis of the MILP model used in CLAASP. Each rule specifies which pairs of input and output exponent vectors are valid, allowing monomial trails to be encoded by linear constraints.

For each operation, a pair $(\mathbf{u}, \mathbf{v})$ is *valid* if the monomial $\mathbf{x}^{\mathbf{u}}$ contributes to $\mathbf{y}^{\mathbf{v}}$ in the algebraic expansion; the set of all such pairs constitutes its propagation rule. Because any Boolean function can be decomposed into XOR, AND, NOT, and COPY, the propagation rules of these basic operations determine all monomial trails. We use binary MILP variables $a_1, a_2, \dots$ for the input exponents and $b, b_1, b_2, \dots$ for the output exponents.

2.5.1 XOR

Let $y = x_1 \oplus x_2 = x_1 + x_2$ over $\mathbb{F}_2$. The algebraic expression of y contains the monomials $\{x_1, x_2\}$. A monomial $x_1^{u_1} x_2^{u_2}$ contributes to y^v if

$$v = u_1 \oplus u_2, \quad (u_1, u_2) \neq (1, 1).$$

Hence, the valid monomial propagations are:

$$x_1^0 x_2^0 \rightarrow y^0, \quad x_1^1 x_2^0 \rightarrow y^1, \quad x_1^0 x_2^1 \rightarrow y^1.$$

The following corresponding exponent transitions define the propagation rules:

$$(0, 0) \rightarrow 0, \quad (1, 0) \rightarrow 1, \quad (0, 1) \rightarrow 1,$$

and that can be captured by the linear equality $b = a_1 + a_2$, which holds for all valid exponent transitions of the XOR operation.

2.5.2 AND

Let $y = x_1 x_2$ over $\mathbb{F}_2$. A monomial $x_1^{u_1} x_2^{u_2}$ contributes to y^v if and only if

$$(u_1, u_2, v) \in \{(0, 0, 0), (1, 1, 1)\},$$

giving the transitions:

$$(0, 0) \rightarrow 0, \quad (1, 1) \rightarrow 1.$$

These are modeled by the linear equalities $b = a_1$, $b = a_2$, ensuring that $b = 1$ exactly when both input exponents are 1.

2.5.3 NOT

Let $y = \neg x = 1 + x$ over $\mathbb{F}_2$. The monomials of y are $\{1, x\}$, giving transitions:

$$0 \rightarrow 0, \quad 0 \rightarrow 1, \quad 1 \rightarrow 1.$$

These are represented by the inequality $b \geq a$, so an active input exponent always activates the output exponent.

2.5.4 COPY

Let $(y_1, y_2) = (x, x)$. The valid exponent transitions are

$$(0) \rightarrow (0, 0), \quad (1) \rightarrow (1, 0), \quad (1) \rightarrow (0, 1), \quad (1) \rightarrow (1, 1).$$

These are modeled by

$$a \geq b_1, \quad a \geq b_2, \quad b_1 + b_2 \geq a,$$

expressing that an active input exponent may activate one or both outputs.

3 Description of CLAASP-MP

This section describes how the 3SDP-woU monomial prediction framework is instantiated inside the CLAASP library. CLAASP-MP is implemented as part of the open-source CLAASP library (<https://github.com/Crypto-TII/claasp>). We first recall the graph-based representation of symmetric primitives in CLAASP, then explain how CLAASP-MP integrates with this representation, and finally describe how basic components are translated into a MILP instance using the propagation rules of Section 2.5.

3.1 Overview of CLAASP-MP

CLAASP-MP is a new analysis module integrated into CLAASP [BGG⁺23]. Its purpose is to provide a unified and fully automated framework for monomial prediction based on the three-subset division property without unknown subset (3SDP-woU). Once a cipher is implemented in CLAASP, CLAASP-MP automatically constructs the corresponding MILP model by encoding the propagation rules of basic components described in Section 2.5 along the directed acyclic graph (DAG) representation of a symmetric primitive.

Using this model, for any selected output bit CLAASP-MP, one can compute the ANF, determine exact and upper bounds on algebraic degree with respect to plaintext, key, or IV variables, and derive the superpoly for a given cube. The same formulation also enables the automated detection of monomial absence and balancedness, providing integral distinguishers directly from the model.

Once the primitive is implemented in CLAASP, no additional modeling effort is required from the user.

3.2 Cipher Representation in CLAASP

In CLAASP, each primitive is a DAG whose nodes are components (S-boxes, linear transformations, Boolean word operations, modular addition, modular subtraction, FSR updates, etc.) and whose edges represent bit connections. Each component has a unique identifier, a type, fixed input and output bit sizes, and a description of how its inputs are taken from the outputs of preceding components. This DAG is sufficient for CLAASP-MP to build its MILP model; every edge is assigned an exponent variable, and each component contributes the linear constraints dictated by its propagation rules. A short illustration of how a primitive is expressed in CLAASP is provided in [Appendix A](#).

Since CLAASP-MP operates directly on this DAG, any primitive implemented in CLAASP is immediately compatible. The library includes a broad collection of almost 100 symmetric primitives.

3.3 MILP Modeling of Components

For a given DAG, CLAASP-MP builds a single MILP model $\mathcal{M}$ by processing components in topological order. Each component contributes to the linear constraints relating its input and output exponent variables.

Boolean word operations are treated at the bit level, linear layers correspond to index mappings or XOR constraints, and nonlinear components such as S-boxes use precomputed monomial transition information derived from their ANF. The next subsections describe MILP constraints for several components.

3.3.1 Boolean word operations.

Boolean word operations such as XOR, AND, NOT, and COPY are modeled at the bit level in CLAASP-MP. Let the component $\mathcal{C}$ have wordsize w , and let (a_i) , (b_i) , (c_i) denote the exponent variables associated with its input and output words. These variables are created once by the MILP model $\mathcal{M}$ according to bit-level connections defined by the DAG. For these operations, the 3SDP-woU propagation rules of [Section 2.5](#) translate into the linear constraints per bit. CLAASP-MP inserts these constraints into a MILP model $\mathcal{M}$ exactly as shown in [algorithm 1](#). For simplicity, we illustrate the two-input case (or the two-output case for COPY); the extension to additional inputs or outputs is straightforward.

Algorithm 1: MILP constraints for Boolean word operations

```

1 XOR component: two input words  $(a_i)$ ,  $(b_i)$ , one output word  $(c_i)$ 
2 for  $i = 0$  to  $w - 1$  do
3    $\mathcal{M}.\text{addConstr}(c_i = a_i + b_i)$ 
4 AND component: two input words  $(a_i)$ ,  $(b_i)$ , one output word  $(c_i)$ 
5 for  $i = 0$  to  $w - 1$  do
6    $\mathcal{M}.\text{addConstr}(c_i = a_i)$ 
7    $\mathcal{M}.\text{addConstr}(c_i = b_i)$ 
8 NOT component: one input word  $(a_i)$ , one output word  $(b_i)$ 
9 for  $i = 0$  to  $w - 1$  do
10   $\mathcal{M}.\text{addConstr}(b_i \geq a_i)$ 
11 COPY component: one input word  $(a_i)$ , two output words  $(b_i)$ ,  $(c_i)$ 
12 for  $i = 0$  to  $w - 1$  do
13   $\mathcal{M}.\text{addConstr}(a_i \geq b_i)$ 
14   $\mathcal{M}.\text{addConstr}(a_i \geq c_i)$ 
15   $\mathcal{M}.\text{addConstr}(b_i + c_i \geq a_i)$ 

```

3.3.2 S-box modeling

CLAASP-MP models an n -bit S-box $F(\mathbf{x}) = (f_0, \dots, f_{n-1})$ via its ANFs. Output exponents v_k link to input monomials $\mathbf{x}^{\mathbf{u}}$ (with $a_{\mathbf{u}}^k = 1$) in the expansion

$$f_k(\mathbf{x}) = \bigoplus_{\mathbf{u}} a_{\mathbf{u}}^k \mathbf{x}^{\mathbf{u}}$$

CLAASP-MP introduces auxiliary binary variables for ANFs monomials. Degree one monomials correspond directly to input exponent variables, while higher degree monomials use a new auxiliary variable constrained to equal each of its degree one factors. These constraints ensure activation only when all inputs in the monomial are active. At a high level, the modeling proceeds as in [algorithm 2](#).

Algorithm 2: MILP constraints for an S-box component

Input: MILP model $\mathcal{M}$, an S-box component

- 1 Compute ANFs of all output bits of the component
- 2 Create auxiliary variables $m_{\mathbf{u}}$ for all monomials $\mathbf{x}^{\mathbf{u}}$ occurring in ANFs
- 3 Link each $m_{\mathbf{u}}$ to its degree-1 factors through linear constraints
- 4 **for each output bit k do**
- 5 form $L_k = \sum_{\mathbf{u}: a_{\mathbf{u}}^k = 1} m_{\mathbf{u}}$
- 6 **if f_k has no constant term then**
- 7 $\mathcal{M}.\text{addConstr}(v_k = L_k)$
- 8 **else**
- 9 $\mathcal{M}.\text{addConstr}(v_k \geq L_k)$

3.3.3 Linear layer modeling

Linear layers $L(\mathbf{x}) = M\mathbf{x}$ with $M \in \mathbb{F}_2^{m \times n}$ are stored in CLAASP's component **description** field. For permutations and rotations, exponent propagation is structural, each output bit is identified with the exponent bit of the input from which it is wired, giving $b_i = a_{\pi(i)}$ in the MILP model.

For general linear layers, an output bit $y_j = \sum_{i: M_{j,i}=1} x_i$ (over $\mathbb{F}_2$) depends on all inputs x_i with a 1 in row j . CLAASP-MP creates for each input exponent a_i as many copies as the number of ones in column i of M , constraining them exactly as in the **COPY** operation. Each occurrence of a_i in an output equation uses a distinct copy ([algorithm 3](#)).

Algorithm 3: MILP constraints for a linear layer

Input: MILP model $\mathcal{M}$, linear component L with matrix M

- 1 **for each input bit i do**
- 2 let w_i = number of ones in column i of M
- 3 create copies $a_i^1, \dots, a_i^{w_i}$: $\mathcal{M}.\text{addConstr}(a_i \geq a_i^\ell)$ for all ℓ ,
- 4 $\mathcal{M}.\text{addConstr}\left(\sum_{\ell=1}^{w_i} a_i^\ell \geq a_i\right)$
- 5 **for each output bit j do**
- 6 initialize an empty linear expression L_j
- 7 **for each input bit i with $M_{j,i} = 1$ do**
- 8 take the next unused copy of a_i , say a_i^ℓ , and set $L_j \leftarrow L_j + a_i^\ell$
- 9 $\mathcal{M}.\text{addConstr}(b_j = L_j)$

This yields a uniform MILP encoding for all linear layers in CLAASP.

3.3.4 Modular multiplication modeling

Unlike standard Boolean word operations, the modeling of modular multiplication is less common in symmetric primitives but is critical to several block ciphers, such as IDEA [LM91] or the recent MSX [MSR⁺26]. While earlier work by Xing et al. [XYL18] explored the division property propagation of modular multiplication using SAT models based on Karnaugh maps, we highlight a different approach.

CLAASP-MP introduces a MILP modeling approach for modular multiplication. Rather than analyzing the multiplication as a single complex algebraic component, our model relies on the exact mathematical expansion of the n -bit modular multiplication $z = (x \times y) \pmod{2^n}$. By expressing both the multiplicand x and the multiplier y in their base-2 representations, $x = \sum_{i=0}^{n-1} x_i 2^i$ and $y = \sum_{j=0}^{n-1} y_j 2^j$, the multiplication can be expanded linearly as:

$$z = \left(\sum_{i=0}^{n-1} x_i 2^i \right) \left(\sum_{j=0}^{n-1} y_j 2^j \right) \pmod{2^n} = \sum_{j=0}^{n-1} \left(\sum_{i=0}^{n-1} (x_i \cdot y_j) 2^{i+j} \right) \pmod{2^n}.$$

This decomposition reduces modular multiplication to a sequence of basic operations on individual bits. The algorithm iterates over each multiplier bit y_j . For each y_j , it computes the scalar products $x_0 y_j, \dots, x_{n-1} y_j$, each modeled as an exact bitwise AND constraint on exponent variables. The factor 2^{i+j} dictates a left-shift by j positions; bits where $i + j > n - 1$ are truncated. The remaining bits form a shifted word s , which is added to the running output z (initialized to 0) via exact modular addition constraints. Iterating over all j accumulates the partial products into the final output $z = (z_0, \dots, z_{n-1})$, as detailed in [algorithm 4](#).

Algorithm 4: MILP constraints for modular multiplication

Input: MILP model $\mathcal{M}$, inputs $x = (x_i)_{i=0}^{n-1}$ and $y = (y_j)_{j=0}^{n-1}$

- 1 Create output word $z = (z_0, \dots, z_{n-1})$ and initialize to 0
- 2 **for** $j = 0$ **to** $n - 1$ **do**
- 3 **for** $i = 0$ **to** $n - 1$ **do**
- 4 Create partial product bit p_i
- 5 $\mathcal{M}.\text{addConstr}(p_i = x_i)$
- 6 $\mathcal{M}.\text{addConstr}(p_i = y_j)$
- 7 Create shifted word $s = (s_0, \dots, s_{n-1})$
- 8 **for** $i = 0$ **to** $n - 1$ **do**
- 9 **if** $i < j$ **then**
- 10 $\mathcal{M}.\text{addConstr}(s_i = 0)$
- 11 **else**
- 12 $\mathcal{M}.\text{addConstr}(s_i = p_{i-j})$
- 13 Update output word z via exact modular addition:
- 14 $z \leftarrow \text{modular_addition}(\mathcal{M}, z, s)$

3.4 MILP Model of the Full Primitive

Once all components of a primitive have been translated into constraints, CLAASP-MP obtains a single MILP model $\mathcal{M}$ describing how exponent variables propagate from the cipher inputs to all cipher outputs. The exponent variables attached to each wire of the DAG are created once and reused across components, so $\mathcal{M}$ captures the complete algebraic structure of the primitive.

All analyses performed by CLAASP-MP are obtained by adding a small number of task-dependent constraints and objectives to this fixed model. To obtain the ANF of an output bit, CLAASP-MP fixes the corresponding output exponent variable to the value one and enumerates every feasible assignment of the input exponent variables. Each feasible pattern corresponds to one monomial appearing in the ANF.

Degree evaluation is performed in the same manner. After fixing an output exponent bit to one, the model maximizes the Hamming weight of the selected input exponent variables by adding a linear objective that maximizes their sum. This yields an upper bound on the algebraic degree. Exact degrees are then recovered by enumerating all feasible monomials of maximal degree and checking their parity; if at least one monomial appears an odd number of times, the exact degree equals this bound, otherwise it is strictly less than this bound. In practice, the degree drops by one, though in principle a further drop could occur if all monomials of the next degree also cancel, which is unlikely and we did not observe it. Cube and superpoly computations also rely on this model. A cube is defined by selecting a set of public inputs as cube variables, while all other public inputs are fixed to zero. The model determines whether the cube monomial reaches the chosen output bit, and the resulting expression in the key variables is the superpoly, with its key-only part given by the key coefficient.

Consequently, every algebraic task supported by CLAASP-MP, including ANF extraction, degree bounds, and cube-based superpoly computation, is performed as a query on a single MILP model built directly from the CLAASP representation of the primitive.

The next section presents experimental results illustrating the use of CLAASP-MP on several symmetric primitives.

4 Experimental Results with CLAASP-MP

In this section, we evaluate the capabilities of CLAASP-MP on block ciphers, permutations, and stream ciphers. All experiments are performed by instantiating the primitives in CLAASP and applying the routines of CLAASP-MP to the common MILP model described in Section 3.4. Also, to help readers reproduce the experiments and to illustrate how the individual routines of CLAASP-MP are invoked in practice, we include in Appendix B a concise set of usage examples (based on SIMON-32) covering ANF extraction, degree computation, cube-based testing, and superpoly computation.

To obtain these results, CLAASP-MP builds upon the algebraic structure of the core MILP model (described in Section 3) by adding objective functions and specific constraints to identify exact degrees, integral distinguishers, and superpolies.

All MILP instances are solved with Gurobi using its default settings. To ensure precession when computing upper bound on degree, we enforce a strict optimality tolerance by setting `OptimalityTol = 1e-9`. In the absence of this adjusted tolerance, the solver may occasionally return false optimal solutions due to floating-point inaccuracies.

Furthermore, we adjust the solver’s search behavior based on the specific cryptanalytic tasks. For tasks that require enumerating all feasible exponent assignments (ANFs, superpolies, key coefficients, and exact degrees), `PoolSearchMode = 2` is used so that the solver returns all solutions satisfying the constraints. To efficiently handle the computational load of enumerating all solutions, CLAASP-MP integrates a simple divide-and-conquer strategy over the MILP model. Tasks that only require a single optimal solution, such as those computing upper bounds on the algebraic degree, use `PoolSearchMode = 0`, which instructs the solver to stop after the first optimal solution and is substantially faster.

4.1 Integral Distinguishers on Block Ciphers

4.1.1 Algebraic Analysis of the MSX Cipher

To demonstrate the practical impact of our new modular multiplication modeling (detailed in Section 3.3), we applied CLAASP-MP to the recently proposed lightweight MSX block cipher [MSR⁺26], which operates on 32-bit words (W_i). We analyze both MSX-64 and MSX-128. Besides the detailed analysis provided by the authors in [MSR⁺26], no other independent algebraic analysis of MSX has been published yet.

Table 1 reports upper bounds on the algebraic degree of MSX for both variants over the first six rounds. For brevity, bounds are grouped by output branch, as all bits within a branch share the same degree growth pattern.

Table 1: Upper bounds on the algebraic degree of the MSX cipher.

Variant	Output Bits	Rounds					
		1	2	3	4	5	6
MSX-64	0–31	32	63	63	63	63	63
	32–63	1	32	63	63	63	63
MSX-128	0–31, 64–95	32	63	94	125	126	126
	32–63, 96–127	1	32	63	94	125	126

As shown in Table 1, the algebraic degree upper bounds of MSX-64 saturate by round 3. For MSX-128, CLAASP-MP identifies 64 output bits with a degree upper bound of 126 at round 6, yielding an integral distinguisher with 2^{127} data complexity.

Beyond degree bounds, we apply CLAASP-MP to identify balanced output bits via monomial prediction. For a cube of dimension d over input bits I , CLAASP-MP maximizes the algebraic degree D of each output bit restricted to the cube variables. If $D < d$, the monomial of degree d is absent from the output ANF, and the corresponding output bit is balanced over all 2^d cube inputs. For MSX-64, any 2-dimensional cube from W_0 leaves all bits in W_1 balanced at round 2; a 33-dimensional cube over bits $\{0–32\}$ leaves W_1 balanced at round 3, consistent with the saturation bounds in Table 1.

For MSX-128, Table 2 summarizes the monomial prediction results. Notably, a 2-dimensional cube in W_0 yields balanced bits in W_1 , W_2 , and W_3 at round 2, which narrows to W_2 and W_3 at round 3, and further to W_3 alone at round 4 all with $\max D \leq 1$. This gradual reduction reflects the progressive, yet still incomplete, inter-word mixing in early rounds. An analogous pattern holds for cubes in W_2 by symmetry, which leaves W_1 balanced at round 4.

Table 2: Monomial prediction in MSX-128: active bits I , cube dimension $d = |I|$, maximum degree D in cube variables, and balanced output words W_j .

Rounds	2	3	4		5	6	7
Cube Bits (I)	$\{0, 1\}$	$\{0, 1\}$	$\{0, 1\}$	$\{0–31\}$	$\{0–32\}$	$\{0–63\}$	W_0, W_2, W_3
Dim (d)	2	2	2	32	33	64	96
Max deg (D)	1	1	1	1	32	63	95
Balanced Words	W_1, W_2, W_3	W_2, W_3	W_3	W_3	W_1	W_1	W_1, W_3

MSX-128 shows a word-aligned monomial structure invariant to the specific bit indices chosen within the cube word. For any 2-dimensional cube from the same word, the balanced outputs depend exclusively on the source word:

- *Round 2.* Cube in W_0 : W_1, W_2, W_3 balanced; in W_1 : W_1, W_3 ; in W_2 : W_0, W_1, W_3 ; in W_3 : W_0, W_1 .
- *Round 3.* Cube in W_0 : W_2, W_3 balanced; in W_1 : W_3 ; in W_2 : W_0, W_1 ; in W_3 : W_1 .
- *Round 4.* Cube in W_0 : W_3 balanced; in W_2 : W_1 balanced. Cubes in W_1 and W_3 no longer yield any balanced word.

The balanced outputs are invariant to the choice of cube bit indices within the source word; only the word itself determines which outputs are balanced. Inter-word mixing remains insufficient until round 5 for low-order cubes. For higher-dimensional cubes, at round 6 the active set $W_0 \cup W_1$ ($d = 64$) leaves W_1 balanced, while $W_2 \cup W_3$ ($d = 64$) leaves W_3 balanced; at round 7, a 96-bit cube still leaves W_1 and W_3 balanced. These findings suggest that further analysis of MSX-128 is warranted, as integral properties persist up to 7 rounds.

4.1.2 Other block ciphers

We next apply CLAASP-MP to search for integral distinguishers across 12 families of block ciphers in CLAASP with different design structures. For each cipher, we inspect whether a chosen cube monomial propagates to a target output bit; an infeasible MILP instance implies that the monomial does not exist in the ANF, thus defining an integral distinguisher. Table 6 in the Appendix summarizes the distinguishers on block ciphers obtained with CLAASP-MP and compares them with the literature. Our tool also reproduces the results for integral distinguishers and superpolies based on parity counting, for example, by simple counting for SIMON and by using a divide-and-conquer strategy for Speck. Some distinguishers obtained by CLAASP-MP that we claimed to match those reported in the literature may appear different at first glance; however, they are in fact equivalent, up to minor differences in the interpretation of the specifications (e.g., endianness conventions).

4.2 Integral Distinguishers on Permutations

We also applied CLAASP-MP to evaluate the algebraic degree growth and to search for integral distinguishers on permutation-based primitives implemented in CLAASP.

For the Ascon and Gaston permutations, we evaluated the upper bound on the algebraic degree across consecutive rounds. The computed upper bounds exactly match the theoretical bounds 2^r for r rounds until round 8, after which the degree saturates to the maximum possible value. Additionally, applying monomial parity counting to 3-round Gaston, CLAASP-MP confirms that many cube monomials of dimension 5, 6, and 7 are absent from the outputs, yielding integral distinguishers.

For ChaCha and Salsa, an integral distinguisher corresponds to an infeasible MILP instance for the cube monomial. CLAASP-MP reproduces the known 6-round distinguishers from the literature. For Salsa, we note that while the 6-round distinguisher reported in [EKKT18] was evaluated on a non-standard implementation, our tool confirms that their proposed cube yields a 5-round distinguisher when applied to the standard Salsa specification. Using the same cube patterns, CLAASP-MP also identifies valid integral distinguishers for higher-round counts than previously reported, reaching up to 6.75 rounds for both ChaCha and Salsa. Table 3 summarizes the maximum-round (6.75) distinguishers found; a detailed list for rounds 6 to 6.75 is given in the Appendix (Table 7).

Table 3: Maximum round (6.75) integral distinguishers identified for ChaCha and Salsa.

Cipher	Rds	Inactive Bits ($\bar{I}$)	Integral Output Bits
ChaCha	6.75	{256}, {288}, {320}, {352}	{96–127, 224–255, 352–383, 480–511}
Salsa	6.75	{18}, {45}, {178}, {205}, {365}, {397}	{359}

4.3 Integral and Algebraic Analysis on Stream Ciphers

We next apply CLAASP-MP to stream ciphers implemented in CLAASP, specifically Bivium and Trivium. Consistent with our previous settings, we identify integral behavior by evaluating the feasibility of the corresponding MILP model. For Bivium, we search

for cubes that exhibit integral behavior, while for Trivium, we examine algebraic-degree growth and superpolies of selected cubes.

Bivium. For Bivium (708 full initialisation clocks), CLAASP-MP identifies several 36-dimensional cubes whose superpolies for the first keystream bit are identically zero, yielding integral distinguishers. Two representative examples are:

$$C_1 = \{i_{14} - i_{30}, i_{35}, i_{43}, i_{54} - i_{70}\}, \quad C_2 = \{i_{15} - i_{31}, i_{36}, i_{44}, i_{55} - i_{71}\}.$$

Many cubes of larger dimension (37–54) also exhibit integral behavior, and for any cube of dimension at least 55, this property holds automatically since the algebraic degree of the first keystream bit reaches 54 after full initialization.

Trivium. For Trivium, we extend the algebraic degree analysis of the first keystream bit beyond the range reported in [HSWW20]. CLAASP-MP computes the *exact* algebraic degree using the routine described in Section 3.4. Table 4 reports the exact degrees obtained beyond the previously analyzed range.

Table 4: Exact degree of Trivium’s first keystream bit beyond 834 clocks.

Clocks	835	836	837	838
Exact degree	78	79	79	79

We also recomputed superpolies for cubes from [DS09] at clocks 672, 735, and 767; all match the published results exactly, confirming correctness. For the cubes from [FV13] at clocks 784 and 799, the superpolies produced by CLAASP-MP differ from those reported, which we attribute to the fact that CLAASP-MP faithfully models Trivium as implemented in CLAASP with all constraints generated automatically.

5 Comparison with Existing Tools

The closest framework to CLAASP-MP is called Solvatore (<https://github.com/kste/solvatore/tree/master>), a SAT-based tool introduced by Eskandari et al. [EKKT18] in 2018, which searches for integral distinguishers by encoding the propagation of the division property into a Boolean satisfiability (CNF) instance. The tool builds on the ideas of Todo and Morii [TM16] from 2016. In this approach, a cipher is represented as a sequence of basic bit-level operations, namely bit-copy, XOR, AND, and the constant bit 1. These form a functionally complete system, allowing Solvatore to express any Boolean function. The tool also includes high-level methods for adding S-boxes, bit permutations, Modular Addition, and MixColumn operations. Each operation type has a predefined SAT encoding, and analyzing a new primitive consists of composing these encodings according to the operation sequence specified for that cipher. Compared to CLAASP, Solvatore is implemented entirely in Python and uses very compact cipher descriptions. Since it targets a specific type of integral distinguisher search based on the bit-based division property, it ignores the key schedule and round-key XOR operations, modeling only the evolution of the internal state. By contrast, CLAASP provides full cipher models, including key scheduling, because its division-property tools support a broader range of applications that require key-dependent behavior. At the time of writing, Solvatore features 40 ciphers, spanning various families, including SPN, ARX, and Feistel.

Solvatore is designed solely for detecting integral distinguishers or to prove their absence. Given a candidate cube, the tool determines whether the corresponding division property vector reaches the target bit, i.e., it checks whether the cube monomial appears in the

ANF, but does not support enumerating monomial parities. In contrast, CLAASP-MP supports a much wider range of algebraic analyses and the refined modeling constraints from [HLM⁺20] (2020, especially for the XOR component), that were not known in 2018 when Solvatore was implemented. From the same MILP model, CLAASP-MP can extract the full ANF of an output bit, compute exact and upper bounds on algebraic degree, determine whether a cube monomial reaches the output, and derive superpolies together with their key coefficients only. Each of these tasks is obtained by adding a small set of query-specific constraints to the same global MILP model, which makes CLAASP-MP a general-purpose tool for monomial prediction over a wide class of symmetric primitives.

A further structural difference between the two frameworks lies in how the underlying algebraic model is constructed. Solvatore builds a separate CNF instance for every candidate cube. In contrast, CLAASP-MP constructs a global MILP model from the CLAASP component graph, independently of the chosen cube, and testing a cube requires only adding a small number of linear constraints on the existing model. This structural difference has a direct impact on practical scalability. For example, while the SAT-based framework reports integral distinguishers for ChaCha and Salsa up to 6 and 5 rounds, respectively, 9, the MILP model underlying CLAASP-MP extends these to 6.75 rounds.

6 Conclusion

We presented CLAASP-MP, a unified MILP-based framework for monomial prediction built on the 3SDP-woU formulation and integrated into the CLAASP library. CLAASP-MP supports ANF extraction, algebraic degree computation, superpoly derivation, key-coefficient recovery, and integral analysis across block ciphers, permutations, hash primitives, and stream ciphers. To demonstrate the effectiveness of the framework, we reproduced several known results. Furthermore, even if the purpose of this work was to introduce the new unified framework, we also demonstrated how CLAASP-MP can be used to produce new original results: we identified new integral cubes for several primitives, extended the published distinguishers for the ChaCha and Salsa permutations, extended the exact algebraic degree computation of Trivium to later initialization clocks, and provided a more fine-grained and improved algebraic analysis of the recently published MSX family of lightweight block ciphers.

Future work includes a more systematic use of CLAASP-MP to existing and newly proposed ciphers to obtain tighter bounds and deeper insights into their algebraic properties. Another direction is to also incorporate recent advances in monomial prediction, such as nested MP, core MP, and refined propagation techniques for nonlinear components developed in [HST⁺21, DL22, HHPW22, HHLW24, HY24], to further improve the computation of large superpolies. Overall, CLAASP-MP provides a practical automated algebraic analysis tool and forms a basis for more precise and scalable monomial-based cryptanalytic techniques.

References

- [BC13] Christina Boura and Anne Canteaut. On the influence of the algebraic degree of f^{-1} on the algebraic degree of $G \circ F$. *IEEE Trans. Inf. Theory*, 59(1):691–702, 2013.
- [BC16] Christina Boura and Anne Canteaut. Another view of the division property. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 654–682. Springer, 2016.
- [BGG⁺23] Emanuele Bellini, David G rault, Juan Grados, Yun Ju Huang, Rusydi H. Makarim, Mohamed Rachidi, and Sharwan K. Tiwari. CLAASP: A Cryptographic Library for the Automated Analysis of Symmetric Primitives. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *Selected Areas in Cryptography - SAC 2023 - 30th International Conference, Fredericton, Canada, August 14-18, 2023, Revised Selected Papers*, volume 14201 of *Lecture Notes in Computer Science*, pages 387–408. Springer, 2023.
- [BRRT24] Emanuele Bellini, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. Mind the composition of toffoli gates: Structural algebraic distinguishers of ARADI. *IACR Cryptol. ePrint Arch.*, page 1559, 2024.
- [BRRT25] Emanuele Bellini, Mohamed Rachidi, Raghvendra Rohit, and Sharwan K. Tiwari. On the structural properties of toffoli gate composition in ARADI: implications for algebraic distinguishers. In Marc Fischlin and Veelasha Moonsamy, editors, *Applied Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23-26, 2025, Proceedings, Part II*, volume 15826 of *Lecture Notes in Computer Science*, pages 400–425. Springer, 2025.
- [Cui19] Cui, Tingting and Wang, Meiqin and Fan, Yanhong and Fu, Yong and Huang, Luning. Ballet. <https://sfjs.cacrnet.org.cn/>, 2019. https://sfjs.cacrnet.org.cn/site/term/list_76_1.html.
- [DKR97] Joan Daemen, Lars R. Knudsen, and Vincent Rijmen. The block cipher square. In Eli Biham, editor, *Fast Software Encryption, 4th International Workshop, FSE ’97, Haifa, Israel, January 20-22, 1997, Proceedings*, volume 1267 of *Lecture Notes in Computer Science*, pages 149–165. Springer, 1997.
- [DL22] Patrick Derbez and Baptiste Lambin. Fast MILP models for division property. *IACR Trans. Symmetric Cryptol.*, 2022(2):289–321, 2022.
- [DS09] Itai Dinur and Adi Shamir. Cube attacks on tweakable black box polynomials. In Antoine Joux, editor, *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*, volume 5479 of *Lecture Notes in Computer Science*, pages 278–299. Springer, 2009.
- [EKKT18] Zahra Eskandari, Andreas Brasen Kidmose, Stefan K lbl, and Tyge Tiessen. Finding integral distinguishers with ease. In Carlos Cid and Michael J. Jacobson Jr., editors, *Selected Areas in Cryptography - SAC 2018 - 25th International Conference, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers*, volume 11349 of *Lecture Notes in Computer Science*, pages 115–138. Springer, 2018.

- [FV13] Pierre-Alain Fouque and Thomas Vannet. Improving key recovery to 784 and 799 rounds of trivium using optimized cube attacks. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 502–517. Springer, 2013.
- [GPRS16] Jian Guo, Thomas Peyrin, Avik Rudra, and Siang Meng Sim. Improved division property based integral attacks on block ciphers, 2016. <https://eprint.iacr.org/2016/857>.
- [HHLW24] Jiahui He, Kai Hu, Hao Lei, and Meiqin Wang. Massive superpoly recovery with a meet-in-the-middle framework - improved cube attacks on trivium and kreyvium. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I*, volume 14651 of *Lecture Notes in Computer Science*, pages 368–397. Springer, 2024.
- [HHPW22] Jiahui He, Kai Hu, Bart Preneel, and Meiqin Wang. Stretching cube attacks: Improved methods to recover massive superpolies. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part IV*, volume 13794 of *Lecture Notes in Computer Science*, pages 537–566. Springer, 2022.
- [HLM⁺20] Yonglin Hao, Gregor Leander, Willi Meier, Yosuke Todo, and Qingju Wang. Modeling for three-subset division property without unknown subset - improved cube attacks against trivium and grain-128aead. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 466–495. Springer, 2020.
- [HST⁺21] Kai Hu, Siwei Sun, Yosuke Todo, Meiqin Wang, and Qingju Wang. Massive superpoly recovery with nested monomial predictions. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part I*, volume 13090 of *Lecture Notes in Computer Science*, pages 392–421. Springer, 2021.
- [HSWW20] Kai Hu, Siwei Sun, Meiqin Wang, and Qingju Wang. An algebraic formulation of the division property: Revisiting degree evaluations, cube attacks, and key-independent sums. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 446–476. Springer, 2020.
- [HW19] Kai Hu and Meiqin Wang. Automatic search for a variant of division property using three subsets. In Mitsuru Matsui, editor, *Topics in Cryptology - CT-RSA 2019 - The Cryptographers’ Track at the RSA Conference 2019, San Francisco, CA, USA, March 4-8, 2019, Proceedings*, volume 11405 of *Lecture Notes in Computer Science*, pages 412–432. Springer, 2019.

- [HY24] Kai Hu and Trevor Yap. Perfect monomial prediction for modular addition. *IACR Trans. Symmetric Cryptol.*, 2024(3):177–199, 2024.
- [Knu94] Lars R. Knudsen. Truncated and higher order differentials. In Bart Preneel, editor, *Fast Software Encryption: Second International Workshop. Leuven, Belgium, 14-16 December 1994, Proceedings*, volume 1008 of *Lecture Notes in Computer Science*, pages 196–211. Springer, 1994.
- [LM91] Xuejia Lai and James L. Massey. A proposal for a new block encryption standard. In Ivan Bjerre Damgård, editor, *Advances in Cryptology — EUROCRYPT ’90*, pages 389–404, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.
- [MSR⁺26] Kazuhiko Minematsu, Tomoyasu Suzaki, Mostafizar Rahman, Sahiba Suryawanshi, and Takanori Isobe. MSX: Lightweight block ciphers for microcontrollers with high-assurance against differential and linear attacks. *IACR Communications in Cryptology*, 2(4), 2026.
- [SPTV22] Roberto La Scala, Sergio Polese, Sharwan K. Tiwari, and Andrea Visconti. An algebraic attack to the bluetooth stream cipher E0. *Finite Fields Their Appl.*, 84:102102, 2022.
- [SPTV24] Roberto La Scala, Federico Pintore, Sharwan K. Tiwari, and Andrea Visconti. A multistep strategy for polynomial system solving over finite fields and a new algebraic attack on the stream cipher trivium. *Finite Fields Their Appl.*, 98:102452, 2024.
- [ST22] Roberto La Scala and Sharwan K. Tiwari. Stream/block ciphers, difference equations and algebraic attacks. *J. Symb. Comput.*, 109:177–198, 2022.
- [TIHM18] Yosuke Todo, Takanori Isobe, Yonglin Hao, and Willi Meier. Cube attacks on non-blackbox polynomials based on division property. *IEEE Trans. Computers*, 67(12):1720–1736, 2018.
- [TM16] Yosuke Todo and Masakatu Morii. Bit-Based Division Property and Application to Simon Family. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 357–377. Springer, 2016.
- [Tod15a] Yosuke Todo. Integral cryptanalysis on full MISTY1. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 413–432. Springer, 2015.
- [Tod15b] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology — EUROCRYPT 2015*, pages 287–314, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [WHG⁺19] Senpeng Wang, Bin Hu, Jie Guan, Kai Zhang, and Tairong Shi. Milp-aided method of searching division property using three subsets and applications. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 398–427. Springer, 2019.

- [XYL18] Yiran Xing, Hailun Yan, and Xuejia Lai. New observation on division property: Simplifying models of basic operations and modeling modular multiplication operation. In Ali Emrouznejad and Zhihong Qian, editors, *The 2nd International Conference on Computer Science and Application Engineering, CSAE 2018, Hohhot, China, October 22-24, 2018*, pages 8:1–8:6. ACM, 2018.
- [XZBL16] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.

A Implementation Examples in CLAASP

This appendix complements [Section 3.2](#) by illustrating CLAASP primitives and their component-based structure used for MILP generation.

A.1 Component Representation: S-box Example

The listing below shows a 4-bit S-box’s internal dictionary: each component includes a unique ID, type, bit sizes, predecessor links, and operation description.

```
# round = 0 - round component = 4
'id': 'sbox_0_4',
'type': 'sbox',
'input_bit_size': 4,
'input_id_link': ['xor_0_1'],
'input_bit_positions': [[0, 1, 2, 3]],
'output_bit_size': 4,
'description': [12, 10, 13, 3, 14, 11, 15, 7, 8, 9, 1, 5, 0, 2, 4, 6],
```

This dictionary fully specifies the S-box’s behavior and connections, providing the only information CLAASP-MP needs to construct its MILP model. By traversing the DAG and applying the rules from [Section 2.5](#), CLAASP-MP attaches exponent variables to connections and generates linear constraints from the S-box ANF.

A.2 Illustration: A Small ToySPN Primitive

[Figure 1](#) illustrates the DAG for a two-round SPN (6-bit block) using two 3-bit S-boxes per round, XOR key injection, and 1-bit left rotation. The cipher structure is shown on the right, with the corresponding CLAASP implementation on the left.

This small example already supports several analysis routines in CLAASP, and through CLAASP-MP it becomes fully compatible with monomial prediction based on the 3SDP-woU model.

B Practical Usage Examples for CLAASP-MP

To illustrate how the routines of CLAASP-MP are used in practice, for clarity and simplicity, the following usage examples are illustrated on the SIMON-32 block cipher as

implemented in CLAASP. CLAASP-MP is implemented within the open-source CLAASP library, available at <https://github.com/Crypto-TII/claasp>. The following examples demonstrate how the main functionalities of CLAASP-MP are invoked in practice.

Available Functionalities

Given any primitive implemented in CLAASP, the monomial prediction model supports the following operations:

- Compute the ANF of a given output bit.
- Determine an upper bound on the algebraic degree of a given output bit.
- Determine the exact algebraic degree of a given output bit.
- Compute the superpoly of a given output bit under a specified cube.
- Determine the exact algebraic degree of such a superpoly.
- Compute an upper bound on the algebraic degree of a cube monomial of a selected output bit.
- Compute the key coefficient of a given cube monomial of a given output bit.

Example Script (SIMON-32)

```
from claasp.ciphers.block_ciphers.simon_block_cipher import SimonBlockCipher
from claasp.cipher_modules.models.milp.milp_models.Gurobi.monomial_prediction \
import MilpMonomialPredictionModel

# Initialize the cipher and the MILP model:
cipher = SimonBlockCipher(number_of_rounds=3)
milp = MilpMonomialPredictionModel(cipher)

# Compute ANF of the output bit at position 0:
milp.find_anf_of_specific_output_bit(0)
Output = p1*p2*p3*p10*p11 + p1*p2*p3*p10 + ... + k52

# Compute upper bound and exact degree of output bits
MILP.find_upper_bound_degree_of_specific_output_bit(0)
Output = 5
milp.find_upper_bound_degree_of_all_output_bits()
Output = [5, 5, ..., 5, 5, 3, 3, ..., 3, 3]
milp.find_exact_degree_of_specific_output_bit(0)
Output = 5
milp.find_exact_degree_of_all_output_bits()
Output = [5, 5, ..., 5, 5, 5, 3, 3, ..., 3, 3]

# Define a cube and compute its related superpoly of the output bit at position 0
cube = ["p16"]
milp.find_superpoly_of_specific_output_bit(0, cube)
Output = p1*p3*p10 + p1*p4*p10*p11 + ... + 1
milp.find_exact_degree_of_superpoly_of_specific_output_bit(0, cube)
Output = 2
milp.find_exact_degree_of_superpoly_of_all_output_bits(cube)
Output = [2, -1, -1, -1, ..., -1, -1, 0, 1]

# Compute degree bounds and coefficients of a cube monomial
MILP.find_upper_bound_degree_of_cube_monomial_of_specific_output_bit(0, cube)
Output = 1
milp.find_keycoeff_of_cube_monomial_of_specific_output_bit(0, cube)
Output = k33*k57 + k50*k57 + k51*k57 + 1
```

Listing 1: Example usage of the monomial prediction model on SIMON-32

C Solvatore vs CLAASP-MP

Table 5 summarizes the main differences between Solvatore [EKKT18] and our tool CLAASP-MP.

Table 5: Comparison between Solvatore and CLAASP-MP

Feature	Solvatore	CLAASP-MP
Modeling approach	SAT, 2SDP [TM16] (2016)	MILP, 3SDP-woU [HLM ⁺ 20] (2020)
Number of available ciphers	40	90
Programming Language	Python	Sagemath/Python
Cipher API	Low-level: Bit-Copy, AND, XOR, Constant bit 1. High-level: S-box, Bit Permutation, MixColumn, ShuffleWords, ModAadd, ModAadd with Constant	S-box, FSR, Constant, Linear Layer, MixColumn, XOR, AND, NOT, OR, ModAdd, ModMul, Rotate, Shift
Cipher Representation	Does not require a key schedule	Needs full cipher implementation
Supported Cryptanalysis Technique	Integral Distinguisher	ANF, Algebraic Degree, Superpoly, Integral Distinguisher,
Integral Distinguisher Approach	Separate CNF instance for every candidate cube	Global MILP model, independent of the chosen cube

D Extended Integral Distinguisher Tables

This appendix provides the comprehensive tables of integral distinguishers obtained by CLAASP-MP across various primitives. Table 6 details the distinguishers identified for block ciphers comparing with established literature results. Table 7 provides the distinguishers found for ChaCha and Salsa permutations, including new higher round distinguishers.

Table 6: Integral distinguishers on block ciphers.

Cipher	Rds	Inactive Bits ($\bar{I}$)	Integral Output Bits	Ref.
Aradi	8	{31, 63, 95, 127}	{32–63, 96–127}	[BRRT24, BRRT25]
Ballet	6	{32}	{127}	[Cui19]
HIGHT	18	{63}	{39}	[EKKT18]
LBlock	17	{30}	{62,63,34,35}	[EKKT18]
LEA	8	{31}	{32–36,55–63,66,118}	[EKKT18]
PRESENT	9	{60,61,62,63}	{63}	[XZBL16]
PRESENT	9	{63}	{3,7,11–15,19,23,27–31,35,39, 43–47, 51,55,59,60–63}	[EKKT18]
SIMON32	13	{0}	{16–31}	[GPRS16]
SIMON48	15	{0}	{24–47}	[GPRS16]
SIMON64	17	{0}	{32–42,48,54–63}	[GPRS16]
SIMON96	21	{0}	{48,50,55,91,96}	[GPRS16]
SIMON128	25	{0}	{64,66,126}	[GPRS16]
SIMECK32	14	{0}	{16,17,21,22,26,27,31}	[GPRS16]
SIMECK48	17	{0}	{24,28,29,43,44}	[GPRS16]
SIMECK64	20	{0}	{32,33,37,59,63}	[GPRS16]
SPECK32	5	{25,26}	{8–15,25–31}	[EKKT18]
SPECK32	6	{26}	{15}	[EKKT18]
PrinceV2	7	{63}	{0}	Ours
Twine	16	{59}	{4}	Ours

```

from claasp.cipher import Cipher

class ToySPN(Cipher):
    def __init__(self):
        super().__init__(family_name="toyspn",
            cipher_type="block_cipher",
            cipher_inputs=["plaintext", "key"],
            cipher_inputs_bit_size=[6, 6],
            cipher_output_bit_size=6)

        sbox = [0, 5, 3, 2, 6, 1, 4, 7]
        self.add_round()
        xor = self.add_XOR_component(
            ["plaintext", "key"],
            [[0,1,2,3,4,5],[0,1,2,3,4,5]],
            6
        )
        sbox1 = self.add_SB0X_component(
            [xor.id],
            [[0, 1, 2]],
            3,
            sbox
        )
        sbox2 = self.add_SB0X_component(
            [xor.id],
            [[3, 4, 5]],
            3,
            sbox
        )
        rotate = self.add_rotate_component(
            [sbox1.id, sbox2.id],
            [[0,1,2],[0,1,2]],
            6,
            1
        )
        self.add_round_output_component(
            [rotate.id],
            [[0,1,2,3,4,5]],
            6
        )

        self.add_round()
        xor = self.add_XOR_component(
            [rotate.id, "key"],
            [[0,1,2,3,4,5],[0,1,2,3,4,5]],
            6
        )
        sbox1 = self.add_SB0X_component(
            [xor.id],
            [[0, 1, 2]],
            3,
            sbox
        )
        sbox2 = self.add_SB0X_component(
            [xor.id],
            [[3, 4, 5]],
            3,
            sbox
        )
        rotate = self.add_rotate_component(
            [sbox1.id, sbox2.id],
            [[0,1,2],[0,1,2]],
            6,
            1
        )
        self.add_cipher_output_component(
            [rotate.id],
            [[0,1,2,3,4,5]],
            6
        )

toyspn = ToySPN()
hex(toyspn.evaluate([0x3F,0x3F]))

```

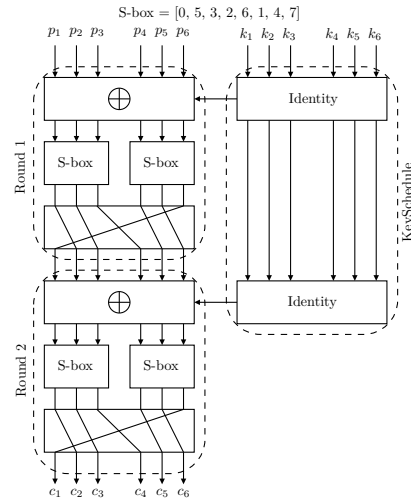


Figure 2: ToySPN structure and implementation.

Table 7: State-of-the-art integral distinguishers for ChaCha and Salsa underlying permutations.

Cipher	Rds	Inactive Bits ($\bar{I}$)	Integral Output Bits	Ref.
ChaCha	6	{0}	{32–68, 192–233, 352–415, 424–428}	[EKKT18]
ChaCha	6	{32}, {160}	{64–100, 224–287, 416–447, 456–460}	Ours
		{64}, {192}	{0–4, 96–159, 288–319, 448–492}	Ours
		{96}, {224}	{0–36, 160–191, 320–396, 480–511}	Ours
		{128}	{32–68, 192–223, 352–415, 424–428}	Ours
		{129}, {289}	{64–66, 424–426}	Ours
		{161}, {321}	{96–98, 456–458}	Ours
		{193}, {353}	{0–2, 488–490}	Ours
		{225}, {257}	{32–34, 392–394}	Ours
ChaCha	6.5	{256}, {288}, {320}, {352}	{0–511}	Ours
		{0}, {128}	{64–68, 192–223, 352–383}	Ours
		{32}	{64–100, 224–255, 456–460}	Ours
		{64}, {192}	{96–127, 448–479, 488–492}	Ours
		{96}	{320–351, 480–511, 488–492}	Ours
		{129}, {289}	{64–66}	Ours
		{160}	{64–100, 224–255, 456–460}	Ours
		{161}, {321}	{96–98, 456–458}	Ours
		{193}, {353}	{488–490}	Ours
		{224}	{320–351, 480–511}	Ours
ChaCha	6.75	{256}, {288}, {320}, {352}	{64–127, 192–255, 320–383, 448–511}	Ours
		{0}, {128}	{352–383}	Ours
		{32}	{96–100, 224–252}	Ours
		{64}, {192}	{96–127, 488–492}	Ours
		{96}, {224}	{480–511}	Ours
		{160}	{96–100, 224–255}	Ours
		{161}, {321}	{96–98}	Ours
		{193}, {353}	{488–490}	Ours
Salsa	6	{256}, {288}, {320}, {352}	{96–127, 224–255, 352–383, 480–511}	Ours
		{0–31}	{73–74, 96–127, 135, 233, 295–304, 448–479}	Ours
		{18}	{199, 359}	Ours
		{45}, {205}, {365}, {397}	{39, 199, 359, 391}	Ours
		{178}	{359, 391}	Ours
Salsa	6.5	{338}	{39, 391}	Ours
		{498}	{39, 199}	Ours
		{18}, {45}, {205}, {365}, {397}	{199, 359}	Ours
		{178}	{359}	Ours
Salsa	6.75	{498}	{199}	Ours
		{18}, {45}, {178}, {205}, {365}, {397}	{359}	Ours